

PROGRAM 6

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/ossp
mkdir -p program6
cd program6
vedsk0831@DESKTOP-83FU7HB:~/ossp/program6$ pwd
/home/vedsk0831/ossp/program6
vedsk0831@DESKTOP-83FU7HB:~/ossp/program6$ nano program6.c
vedsk0831@DESKTOP-83FU7HB:~/ossp/program6$ gcc -Wall -Wextra -g program6.c -o program6
vedsk0831@DESKTOP-83FU7HB:~/ossp/program6$ ls
program6  program6.c
vedsk0831@DESKTOP-83FU7HB:~/ossp/program6$ ./program6
=====
      OOSP PROGRAM 6
  Escape Sequences & Process Execution
=====

Examples:
  ls -l
  echo hello
  echo hello\ world
  pwd
  date

Type 'exit' to quit.

shell6> echo hello

Parsed arguments:
  argv[0] = [echo]
  argv[1] = [hello]
Parent PID: 789
Child PID: 790
Parent waiting for child...
```

```
Child process created.  
Child PID: 790  
hello  
Child exited with status: 0
```

```
shell6> echo hello world
```

```
Parsed arguments:  
  argv[0] = [echo]  
  argv[1] = [hello]  
  argv[2] = [world]  
Parent PID: 789  
Child PID: 801  
Parent waiting for child...
```

```
Child process created.  
Child PID: 801  
hello world  
Child exited with status: 0
```

```
shell6> echo hello\ world
```

```
Parsed arguments:  
  argv[0] = [echo]  
  argv[1] = [hello world]  
Parent PID: 789  
Child PID: 804  
Parent waiting for child...
```

```
Child process created.  
Child PID: 804  
hello world
```

Child exited with status: 0

shell6> ls -l

Parsed arguments:

argv[0] = [ls]

argv[1] = [-l]

Parent PID: 789

Child PID: 805

Parent waiting for child...

Child process created.

Child PID: 805

total 32

-rwxr-xr-x 1 vedsk0831 vedsk0831 20736 Sep 17 04:51 program6

-rw-r--r-- 1 vedsk0831 vedsk0831 4711 Sep 17 04:50 program6.c

Child exited with status: 0

shell6> pwd

Parsed arguments:

argv[0] = [pwd]

Parent PID: 789

Child PID: 806

Parent waiting for child...

Child process created.

Child PID: 806

/home/vedsk0831/ossdp/program6

Child exited with status: 0

shell6> abcdef123

```
Parsed arguments:
  argv[0] = [abcdef123]
Parent PID: 789
Child PID: 807
Parent waiting for child...

Child process created.
Child PID: 807
execvp: No such file or directory
Child exited with status: 1

shell6> valgrind --leak-check=full --show-leak-kinds=all ./program6

Parsed arguments:
  argv[0] = [valgrind]
  argv[1] = [--leak-check=full]
  argv[2] = [--show-leak-kinds=all]
  argv[3] = [./program6]
Parent PID: 789

Child PID: 808
Child process created.
Parent waiting for child...
Child PID: 808
==808== Memcheck, a memory error detector
==808== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==808== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==808== Command: ./program6
==808==
=====
                OSSP PROGRAM 6
    Escape Sequences & Process Execution
```

Examples:

```
ls -l
echo hello
echo hello\ world
pwd
date
```

Type 'exit' to quit.

```
shell6> echo hello
```

Parsed arguments:

```
argv[0] = [echo]
argv[1] = [hello]
```

Parent PID: 808

Child PID: 809

Parent waiting for child...

Child process created.

Child PID: 809

hello

Child exited with status: 0

```
shell6> echo hello\world
```

Parsed arguments:

```
argv[0] = [echo]
argv[1] = [helloworld]
```

Parent PID: 808

Child PID: 812

Parent waiting for child...

```

Child process created.
Child PID: 812
helloworld
Child exited with status: 0

shell6> pwd

Parsed arguments:
  argv[0] = [pwd]
Parent PID: 808
Child PID: 813
Parent waiting for child...

Child process created.
Child PID: 813
/home/vedsk0831/ossdp/program6
Child exited with status: 0

shell6> exit
Program 6 terminated.
==808==
==808== HEAP SUMMARY:
==808==      in use at exit: 0 bytes in 0 blocks
==808==    total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==808==
==808== All heap blocks were freed -- no leaks are possible
==808==
==808== For lists of detected and suppressed errors, rerun with: -s
==808== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
Child exited with status: 0

shell6> |

```

C program:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


#define MAX_INPUT 1024

#define MAX_ARGS 100

```

```

/*
 * Parse command and handle escaped characters.
 *
 * Example:
 *
 * echo hello world
 *
 * becomes:
 * argv[0] = echo
 * argv[1] = hello
 * argv[2] = world
 *
 * echo hello\ world
 *
 * becomes:
 * argv[0] = echo
 * argv[1] = hello world
 */

```

```

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && argc < MAX_ARGS - 1)
    {
        /* Skip spaces */
        while (input[i] == ' ' || input[i] == '\t')
        {

```

```
    i++;  
}
```

```
if (input[i] == '\0')  
{  
    break;  
}
```

```
/* Start of a new argument */  
args[argc++] = &input[i];
```

```
int write_pos = i;
```

```
while (input[i] != '\0')  
{  
    /*  
     * Backslash escape  
     *  
     * hello\ world  
     *  
     * becomes  
     *  
     * hello world  
     */  
    if (input[i] == '\\' && input[i + 1] != '\0')  
    {  
        i++;
```

```

        input[write_pos++] = input[i++];
    }

    /*
     * Normal character
     */
    else if (input[i] != ' ' && input[i] != '\t')
    {
        input[write_pos++] = input[i++];
    }

    /*
     * Normal space means
     * end of current argument.
     */
    else
    {
        i++;
        break;
    }
}

/* Terminate argument */
input[write_pos] = '\0';
}

/* exec requires NULL at the end */
args[argc] = NULL;

```

```
    return argc;
}
```

```
int main()
```

```
{
```

```
    char input[MAX_INPUT];
```

```
    char *args[MAX_ARGS];
```

```
    printf("=====\n");
```

```
    printf("    OSSP PROGRAM 6\n");
```

```
    printf("  Escape Sequences & Process Execution\n");
```

```
    printf("=====\n");
```

```
    printf("\nExamples:\n");
```

```
    printf("  ls -l\n");
```

```
    printf("  echo hello\n");
```

```
    printf("  echo hello\\ world\n");
```

```
    printf("  pwd\n");
```

```
    printf("  date\n");
```

```
    printf("\nType 'exit' to quit.\n\n");
```

```
while (1)
```

```
{
```

```
    printf("shell6> ");
```

```
fflush(stdout);
```

```
/*
```

```
 * Read command
```

```
*/
```

```
if (fgets(input, sizeof(input), stdin) == NULL)
```

```
{
```

```
    break;
```

```
}
```

```
/*
```

```
 * Remove newline
```

```
*/
```

```
input[strcspn(input, "\n")] = '\0';
```

```
/*
```

```
 * Ignore empty input
```

```
*/
```

```
if (strlen(input) == 0)
```

```
{
```

```
    continue;
```

```
}
```

```
/*
```

```
 * Exit
```

```
*/
```

```
if (strcmp(input, "exit") == 0)
```

```
{
```

```
        break;
    }

    /*
     * Parse command
     */
    int argc = parse_command(input, args);

    if (argc == 0)
    {
        continue;
    }

    /*
     * Display arguments
     */
    printf("\nParsed arguments:\n");

    for (int i = 0; i < argc; i++)
    {
        printf(" argv[%d] = [%s]\n", i, args[i]);
    }

    /*
     * Create child process
     */
    pid_t pid = fork();
```

```
if (pid < 0)
```

```
{
```

```
    perror("fork");
```

```
    continue;
```

```
}
```

```
/*
```

```
 * CHILD PROCESS
```

```
*/
```

```
if (pid == 0)
```

```
{
```

```
    printf("\nChild process created.\n");
```

```
    printf("Child PID: %d\n", getpid());
```

```
/*
```

```
 * Execute command.
```

```
 *
```

```
 * execvp searches PATH automatically.
```

```
*/
```

```
execvp(args[0], args);
```

```
/*
```

```
 * Reached only if execvp fails.
```

```
*/
```

```
perror("execvp");
```

```

    exit(EXIT_FAILURE);
}

/*
 * PARENT PROCESS
 */
else
{
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent waiting for child...\n");

    int status;

    /*
     * Wait for child to finish
     */
    waitpid(pid, &status, 0);

    /*
     * Check child status
     */
    if (WIFEXITED(status))
    {
        printf(
            "Child exited with status: %d\n",

```

```
        WEXITSTATUS(status)

    );

}

else

{

    printf("Child terminated abnormally.\n");

}

}

printf("\n");

}

printf("Program 6 terminated.\n");

return 0;

}
```